Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №6

Дисциплина: Основы программирования на Python

Тема: Декораторы, контекстные менеджеры и расширенные возможности Python

Вариант 2: Библиотека

Цель работы

Получить практические навыки применения расширенных возможностей языка Python при разработке приложений: классов данных dataclass, перечислений Enum, декораторов, контекстных менеджеров, протоколов typing.Protocol и механизма интроспекции объектов. В ходе работы необходимо научиться выносить сквозную функциональность в декораторы, управлять ресурсами через контекстные менеджеры и описывать архитектурные контракты без наследования.

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать систему управления библиотечным фондом. Объекты предметной области: книга, журнал и электронное издание. Статусы объектов: доступно, выдано, зарезервировано. Дополнительные операции: выдача издания, возврат издания, поиск просроченных выдач. Приложение должно содержать несколько типов объектов, перечисление состояний на основе Enum, классы данных на основе dataclass, аннотации типов, собственные декораторы (в том числе декоратор с параметрами), собственный контекстный менеджер, Protocol, механизм интроспекции, консольное меню, обработку исключений и формирование итоговой статистики.

Краткие теоретические сведения

Декоратор dataclass автоматически создаёт конструктор, методы сравнения и представления для класса данных, что избавляет от шаблонного кода. Перечисление Enum задаёт фиксированный набор именованных состояний; обращение к атрибуту value возвращает строковое представление статуса. Аннотации типов не влияют на выполнение программы, но документируют контракт и позволяют статическим анализаторам находить ошибки.

Декоратор — это функция, которая принимает другую функцию и возвращает её улучшенную версию. Внутри создаётся функция-обёртка, вызывающая исходную функцию и добавляющая поведение до или после вызова. Декоратор functools.wraps копирует в обёртку имя и строку документации исходной функции, благодаря чему не нарушается интроспекция. Декоратор с параметрами реализуется как функция, возвращающая декоратор: внешний слой принимает параметры, средний — саму функцию, внутренний — аргументы вызова.

Контекстный менеджер гарантирует выполнение завершающих действий при выходе из блока with. Простейший способ его создать — декоратор contextlib.contextmanager над функцией-генератором: код до оператора yield выполняется при входе в блок, код после yield — при успешном выходе, а обработка исключений вокруг yield позволяет реализовать откат изменений.

Protocol из модуля typing описывает структурный интерфейс: класс соответствует протоколу, если в нём есть методы с нужными сигнатурами, явное наследование не требуется. Интроспекция — это исследование объекта во время выполнения: функции type(), dir(), isinstance(), атрибуты __class__ и модуль inspect позволяют получить тип объекта, список его атрибутов и сигнатуры методов.

Структура проекта

Файл	Содержимое
models.py	Перечисление LibraryStatus, dataclass LibraryEdition и производные Book, Magazine, DigitalEdition
decorators.py	Декораторы log_call, measure_time и limit_calls с параметром
context.py	Контекстный менеджер transaction_mode на основе contextmanager
protocols.py	Протокол Reportable с методом get_report_data()
exceptions.py	Иерархия исключений LibraryError, EntityNotFoundError, InvalidStateError
services.py	Класс LibraryService с бизнес-логикой и функция интроспекции inspect_object()
main.py	Консольное меню из двенадцати пунктов и обработка исключений

Модель данных: Enum и dataclass

Состояние издания описывается перечислением LibraryStatus с тремя значениями: AVAILABLE («доступно»), ISSUED («выдано») и RESERVED («зарезервировано»). Использование Enum исключает опечатки в строковых статусах и делает проверки в бизнес-логике наглядными.

Базовый класс LibraryEdition объявлен как dataclass и содержит идентификатор, название, статус и срок возврата. Внутренний рейтинг издания хранится в поле _internal_rating, которое исключено из конструктора параметром init=False, а доступ к нему организован через свойство rating с сеттером, проверяющим диапазон от 0.0 до 10.0. При выходе за границы возбуждается исключение ValueError. Производные классы Book, Magazine и DigitalEdition добавляют собственные поля: автора и число страниц, номер выпуска и месяц, формат файла и его размер.

Листинг 6.1 — Перечисление статусов и базовый dataclass (models.py)

```
class LibraryStatus(Enum):
    AVAILABLE = "доступно"
    ISSUED = "выдано"
    RESERVED = "зарезервировано"


@dataclass
class LibraryEdition:
    id: int
    title: str
    status: LibraryStatus = LibraryStatus.AVAILABLE
```

```
    due_date: Optional[date] = None
    _internal_rating: float = field(default=5.0, init=False)

    @property
    def rating(self) -> float:
        return self._internal_rating

    @rating.setter
    def rating(self, value: float) -> None:
        if not (0.0 <= value <= 10.0):
            raise ValueError("Рейтинг должен быть в диапазоне от 0.0 до 10.0")
        self._internal_rating = value
```

Декораторы

В модуле decorators.py реализованы три декоратора. Первый, log_call, выводит имя вызванного метода, его аргументы и результат, а при возникновении исключения фиксирует тип ошибки и пробрасывает её дальше. Второй, measure_time, замеряет длительность выполнения с помощью time.perf_counter() и выводит время в секундах. Третий, limit_calls, принимает параметр max_calls и ограничивает число допустимых вызовов метода, возбуждая RuntimeError при превышении лимита. Счётчик хранится в замыкании и изменяется через объявление nonlocal. Все три обёртки декорированы functools.wraps, поэтому имя и документация исходных методов сохраняются.

Декораторы применяются к методам сервиса: добавление и удаление изданий журналируются, поиск по идентификатору измеряется по времени, а метод получения статистики ограничен пятью вызовами. Допускается наложение декораторов, например метод выдачи обёрнут одновременно в log_call и measure_time.

Листинг 6.2 — Декоратор с параметром (decorators.py)

```
def limit_calls(max_calls: int) -> Callable[..., Any]:
    def decorator(func: Callable[..., Any]) -> Callable[..., Any]:
        calls = 0

        @wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            nonlocal calls
            if calls >= max_calls:
                raise RuntimeError(
                    f"[LIMIT] Превышен лимит вызовов ({max_calls}) "
                    f"для метода '{func.__name__}'.")
            calls += 1
```

```
            return func(*args, **kwargs)
        return wrapper
    return decorator
```

Контекстный менеджер

Функция transaction_mode(), оформленная декоратором contextmanager, имитирует транзакционный режим работы с коллекцией. При входе в блок with выводится сообщение о начале транзакции, при успешном завершении — сообщение о фиксации изменений, при возникновении исключения внутри блока выполняется откат и ошибка пробрасывается дальше, а блок finally сообщает о закрытии сессии. В консольном меню этот менеджер используется для демонстрации: пользователь может вызвать искусственную ошибку и увидеть сообщение об откате.

Листинг 6.3 — Контекстный менеджер (context.py)

```
@contextmanager
def transaction_mode(collection_name: str) -> Generator[None, None, None]:
    print(f"--- [TRANSACTION START] Открыт режим для: '{collection_name}' ---")
    try:
        yield
        print(f"--- [TRANSACTION SUCCESS] Операции зафиксированы ---")
    except Exception as e:
        print(f"--- [TRANSACTION ROLLBACK] Ошибка: {type(e).__name__}. "
              f"Откат изменений! ---")
        raise
    finally:
        print(f"--- [TRANSACTION END] Сессия закрыта ---")
```

Протокол и полиморфизм

Протокол Reportable объявляет единственный метод get_report_data(), возвращающий строковое описание издания. Каждый производный класс реализует этот метод по-своему, поэтому метод сервиса print_protocol_reports() может принимать список объектов и вызывать общий метод, не проверяя конкретный тип. Таким образом структурная типизация заменяет наследование от абстрактного класса и позволяет подключать новые типы изданий без изменения сервиса.

Механизм интроспекции

Функция inspect_object() демонстрирует пять приёмов исследования объекта: получение точного типа через type(), имени класса через атрибут __class__.__name__, проверку принадлежности базовому классу через isinstance(), перечень атрибутов и методов через dir() с фильтрацией служебных имён, а также чтение сигнатур методов через inspect.getmembers() и inspect.signature(). В консольном меню интроспекция применяется к объекту сервиса, что позволяет увидеть его состояние и полный набор методов с параметрами.

Листинг 6.4 — Интроспекция объекта (services.py)

```
def inspect_object(obj: Any) -> None:
    print("1. Тип (type):", type(obj))
    print("2. Имя класса (__class__):", obj.__class__.__name__)
    print("3. isinstance(obj, LibraryEdition):", isinstance(obj, LibraryEdition))
    print("4. Атрибуты и методы:", [a for a in dir(obj) if not a.startswith('__')])


    for name, method in inspect.getmembers(obj, predicate=inspect.ismethod):
        try:
            print(f"   Метод: {name}{inspect.signature(method)}")
        except Exception:
            pass
```

Раздел тестирования

Проверка выполнялась вручную через консольное меню. Перед началом работы в фонд загружаются три издания: книга, журнал и книга с искусственно просроченной датой возврата, что позволяет сразу проверить поиск задолженностей. Отдельно проверялись граничные случаи: повторная выдача уже выданного издания, возврат издания, находящегося на полке, и превышение лимита вызовов статистики.

№	Проверяемое действие	Результат
1	Добавление книги, журнала и электронного издания	Все три объекта созданы, в логе отражены вызовы add_edition()
2	Вывод всех изданий через Protocol	Для каждого объекта вызван собственный get_report_data()
3	Поиск издания по ID	Объект найден, в консоль выведено время выполнения метода

№	Проверяемое действие	Результат
4	Выдача доступного издания на 14 дней	Статус изменён на «выдано», рассчитана дата возврата
5	Выдача уже выданного издания	Возбуждено InvalidStateError, выведено сообщение об ошибке бизнес-логики
6	Возврат издания	Статус изменён на «доступно», дата возврата очищена
7	Возврат издания со статусом «доступно»	Возбуждено InvalidStateError
8	Поиск просроченных выдач	Найдена книга с датой возврата на пять дней раньше текущей
9	Получение статистики	Выведены общий фонд, доступные, выданные, зарезервированные и просроченные
10	Шестой вызов статистики подряд	Возбуждено RuntimeError от декоратора limit_calls
11	Интроспекция сервиса	Выведены тип, имя класса, атрибуты и сигнатуры методов
12	Транзакция с имитацией сбоя	Выведены сообщения START, ROLLBACK и END, ошибка обработана в меню

Проверка ввода некорректных данных подтвердила работу обработки исключений: при вводе текста вместо числового идентификатора перехватывается ValueError, при обращении к несуществующему идентификатору — EntityNotFoundError, а любая непредвиденная ошибка выводится с указанием типа и сообщения. Программа при этом не завершается.

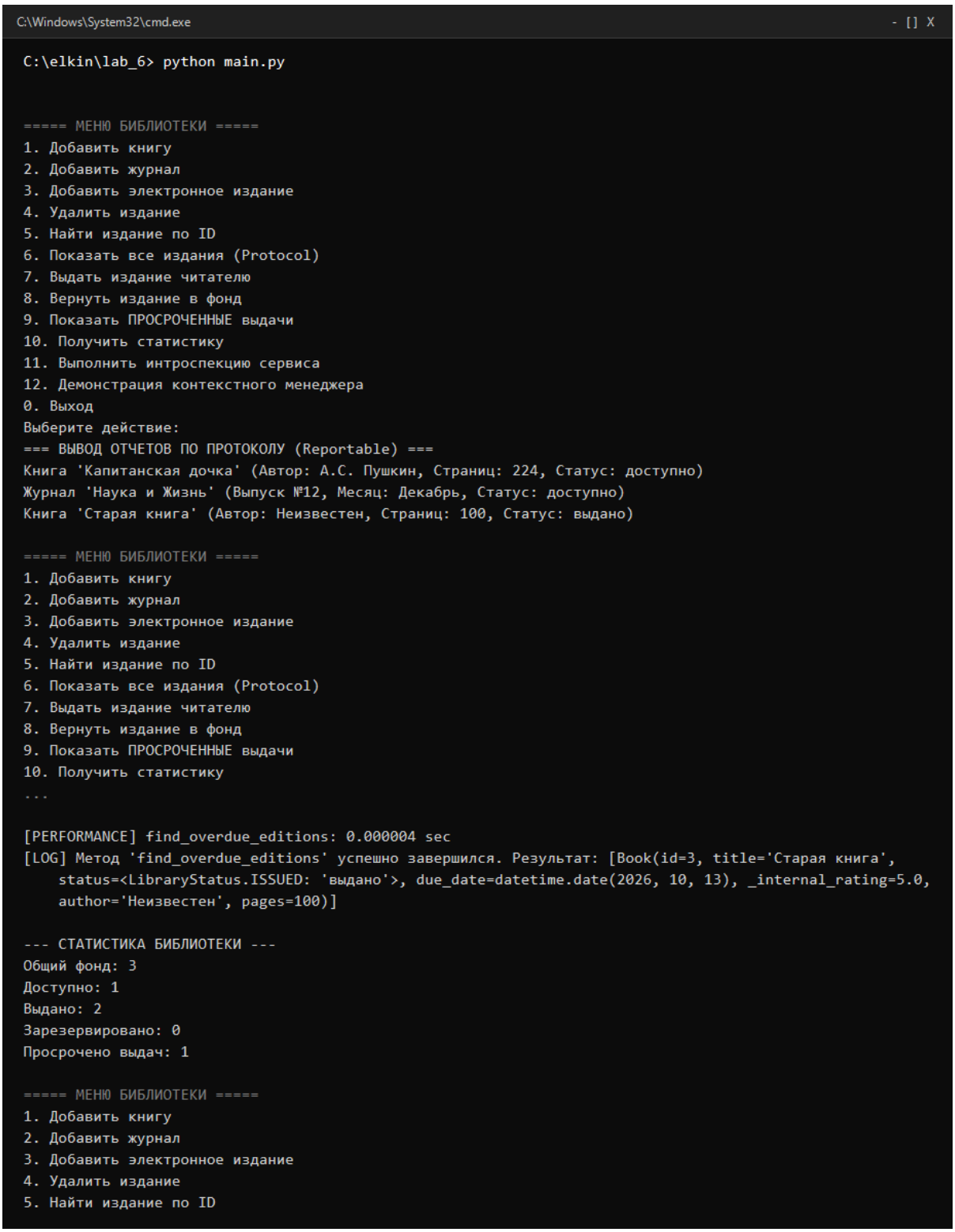


Рисунок 6.1 — Работа программы: отчёты по протоколу, меню библиотеки и статистика фонда

Листинги ключевых фрагментов программы

Листинг 6.5 — Протокол Reportable (protocols.py)

```
from typing import Protocol
```

```
class Reportable(Protocol):
    def get_report_data(self) -> str:
        ...
```

Листинг 6.6 — Наложение декораторов и бизнес-операции (services.py)

```
    @log_call
    @measure_time
    def issue_edition(self, edition_id: int, days_to_borrow: int = 14) -> str:
        edition = self.find_by_id(edition_id)
        if edition.status != LibraryStatus.AVAILABLE:
            raise InvalidStateError(
                f"Невозможно выдать. Текущий статус: '{edition.status.value}'.")
        edition.status = LibraryStatus.ISSUED
        edition.due_date = date.today() + timedelta(days=days_to_borrow)
        return f"Издание '{edition.title}' выдано. Срок возврата: {edition.due_date}"
```

Листинг 6.7 — Поиск просроченных выдач и статистика (services.py)

```
    def find_overdue_editions(self) -> List[LibraryEdition]:
        current_date = date.today()
        overdue = [
            ed for ed in self.editions
            if ed.status == LibraryStatus.ISSUED and ed.due_date
            and ed.due_date < current_date
        ]
        return overdue


    @limit_calls(max_calls=5)
    def get_statistics(self) -> dict[str, Any]:
        total = len(self.editions)
        available = sum(1 for x in self.editions
                        if x.status == LibraryStatus.AVAILABLE)
        issued = sum(1 for x in self.editions
                     if x.status == LibraryStatus.ISSUED)
        reserved = sum(1 for x in self.editions
                       if x.status == LibraryStatus.RESERVED)
        overdue_count = len(self.find_overdue_editions())
        return {"Общий фонд": total, "Доступно": available,
                "Выдано": issued, "Зарезервировано": reserved,
                "Просрочено выдач": overdue_count}
```

Вывод

В ходе выполнения лабораторной работы разработано консольное приложение управления библиотечным фондом, построенное на расширенных возможностях Python. Применение dataclass и Enum позволило описать модель данных без шаблонного кода и свести состояния изданий к конечному набору значений. Декораторы вынесли журналирование, замер времени и ограничение числа вызовов из бизнес-логики, контекстный менеджер обеспечил единообразную обработку транзакционных ситуаций, Protocol задал общий контракт для разных типов изданий, а интроспекция дала возможность

исследовать объекты во время выполнения. Полученные навыки позволяют писать более компактный и расширяемый код.